

1. Introduction

1.1 Purpose

The purpose of this document is to describe the Software Requirements Specification (SRS) for the AI-Based Layout Planning and Price Prediction System. This document provides a detailed explanation of the system requirements, functionalities, constraints, and expected behavior of the software system. It acts as a guideline for developers, project managers, testers, and stakeholders involved in the development of the system.

The AI-Based Layout Planning and Price Prediction System is a desktop application that combines AI-driven architectural floor plan generation with machine learning-based real estate price prediction. The system allows users to input plot dimensions, number of rooms, and facing direction to automatically receive a 2D floor plan and a predicted property price. The platform is built using a PySide6 desktop frontend, a FastAPI backend, and two independent ML components: a LayoutModel and a PricePrediction Model.

This document defines both the functional requirements (what the system should do) and non-functional requirements (how the system should perform). It also describes system interfaces, operating environments, constraints, and assumptions related to the development and use of the software.

1.2 Document Conventions

This Software Requirements Specification follows a structured documentation format in order to clearly describe the system requirements. Each section of the document is organized with numbered headings to maintain clarity and consistency throughout the document.

Important requirement statements are written using formal language such as "shall", "must", and "should". The word "shall" is used to represent mandatory system requirements that must be implemented. The word "should" indicates recommended functionality that improves the system but may not be essential for the initial version.

Requirements are uniquely identified using requirement identification numbers such as REQ-F-01, REQ-F-02, REQ-NF-01, and so on. These identifiers help developers and testers easily trace and verify individual requirements during system development and testing.

In cases where complete information is not available at the time of writing this document, the term TBD (To Be Determined) is used. These sections will be updated once the required information becomes available.

1.3 Intended Audience and Reading Suggestions

This document is intended for several groups of readers who are involved in the design, development, testing, and use of the system. Each group may refer to different sections of the document depending on their responsibilities.

Software developers will use this document to understand the system architecture, functionalities, and technical requirements required for implementing the PySide6 UI, FastAPI backend, and ML components. They will refer to sections describing system features, interfaces, and constraints.

Project managers will use this document to monitor project scope, schedule, and progress. It will help them ensure that the development process remains aligned with the defined system objectives.

Test engineers will use the requirements described in this document to design test cases and verify whether the system functions correctly according to the specifications.

ML Engineers and data scientists will use the document to understand the requirements for the LayoutModel and PricePrediction Model, including input parameters, output expectations, and model accuracy constraints.

End users such as homeowners, property developers, and architects will refer to the document to understand the capabilities of the system and how it can help them in layout planning and price estimation.

Readers are recommended to begin with the Introduction section to understand the overall objective of the system. After that, they should proceed to the Overall Description section to understand the system context and environment. The System Features section should be read next to understand the detailed functionalities of the system.

1.4 Product Scope

The AI-Based Layout Planning and Price Prediction System is a desktop software application developed to automate architectural floor plan generation and real estate price estimation for residential plots. The system accepts plot dimensions, room configuration, and facing direction as inputs and produces an AI-generated 2D floor plan along with a predicted property price.

The primary goal of the system is to help property developers, homeowners, and architects reduce planning time and make data-driven investment decisions. By using ML-based layout optimization and regression-based price prediction, the system delivers results that would traditionally require expert consultation.

The platform will provide an interactive desktop interface built with PySide6 that allows users to view the generated floor plan, explore room arrangements, and understand the cost breakdown. The system also supports Vastu-compliant layout generation and layout constraint validation with automatic correction.

The system will operate fully offline and will support PDF export of floor plans and cost reports. The platform serves as an intelligent planning tool that combines AI, machine learning, and architectural knowledge to support real estate decisions.

2. Overall Description

2.1 Product Perspective

The AI-Based Layout Planning and Price Prediction System is a standalone desktop application designed to operate as an intelligent planning layer for residential real estate projects. The system is not intended to replace professional architectural software but rather to provide an AI-powered first-draft generation capability accessible to non-technical users.

The system collects plot data from the user through the PySide6 desktop interface and passes it to the FastAPI backend via HTTP requests. The backend triggers two independent ML models: the LayoutModel processes spatial data to generate optimal room placement, and the PricePrediction Model estimates the property value based on built-up area and material grade.

The overall system architecture includes three tiers: the PySide6 Frontend UI layer containing Qt Widgets and a Matplotlib canvas; the FastAPI Backend layer handling API request routing and ML model invocation; and the ML Components layer containing the LayoutModel and PricePrediction Model. All inter-layer communication uses JSON format over HTTP. This architecture ensures modularity and allows each component to be independently updated.

2.2 Product Functions

The system performs several important functions related to layout generation, price prediction, validation, and visualization. Initially, the system allows users to input plot parameters such as dimensions, number of rooms, and facing direction. Once the input is provided, the system sends an API request to the FastAPI backend which triggers both ML models simultaneously.

The LayoutModel processes the plot data and generates room coordinates and dimensions for all specified rooms. It applies Vastu Shastra directional correction rules based on the specified facing direction and places furniture icons and car parking layout in the plan.

The PricePrediction Model calculates the total built-up area and applies cost-per-sqft rates based on the selected material grade to produce a detailed cost breakdown and a total predicted property price.

The system also provides an interactive desktop interface that displays the generated floor plan through an embedded Matplotlib canvas. The interface allows users to view separate tabs for each floor and export the complete plan with cost report as a PDF file.

2.3 User Classes and Characteristics

The system will be used by several types of users, each having different roles and responsibilities.

The primary user class is the Homeowner or Property Buyer, who interacts with the system to generate floor plans for their residential plot and understand the estimated construction cost. These users are typically non-technical and rely on the visual output of the system to make decisions.

The second user class is the Property Developer or Real Estate Professional, who uses the system to quickly generate multiple layout options for client presentations. These users have moderate technical knowledge and benefit from the Vastu compliance and cost estimation features.

The third user class is the Architect or Technical User, who uses the system as a rapid prototyping tool to generate initial floor plan drafts. These users have high technical knowledge and may use the PDF export to refine layouts in professional CAD software.

Each of these user groups interacts with the system in different ways depending on their roles and technical expertise.

2.4 Operating Environment

The AI-Based Layout Planning and Price Prediction System will operate on standard desktop and laptop computers with common software environments. The system supports Windows 10/11 and Linux (Ubuntu 20.04+) operating systems.

The software will be developed using the Python programming language along with libraries such as PySide6 for the desktop interface, FastAPI for the backend server, Scikit-learn for machine learning models, and Matplotlib for 2D floor plan rendering. SQLite will be used for local data storage.

The hardware requirements for running the system include a computer with at least 4 GB of RAM, a dual-core processor at 2.0 GHz or higher, and 2 GB of available storage space. No internet connection is required as all processing is performed locally.

2.5 Design and Implementation Constraints

Several design and implementation constraints must be considered during system development. The system shall be implemented using Python 3.10+ as the primary language due to its machine learning library ecosystem. The desktop UI shall be built using PySide6 for cross-platform compatibility on Windows and Linux.

The floor plan drawing engine shall use Matplotlib only, without dependency on external CAD libraries. All communication between the frontend and backend shall use JSON format exclusively. The system shall not require an internet connection for any core functionality.

Additionally, the system must be designed in a modular manner so that future improvements or feature additions can be implemented easily without affecting existing functionalities.

2.6 User Documentation

The system will be accompanied by several forms of user documentation to help users understand how to operate the platform effectively.

A User Manual will be provided explaining how to install the system, enter plot parameters, generate floor plans, and interpret the predicted price output. An Installation Guide will assist users in setting up the Python environment, installing dependencies, and starting the FastAPI backend server.

A Quick Reference Card will be included summarizing the input fields, button functions, and export options available in the PySide6 interface. Technical documentation will also be available for developers covering the FastAPI endpoint specifications, ML model interfaces, and database schema.

2.7 Assumptions and Dependencies

Certain assumptions have been made while designing the system. It is assumed that users will have access to a Windows or Linux desktop computer meeting the minimum hardware requirements. It is also assumed that users possess basic computer knowledge and are familiar with desktop application usage.

The system depends on several external Python libraries including PySide6, FastAPI, Scikit-learn, Matplotlib, Shapely, SQLAlchemy, and ReportLab. If these libraries are unavailable or incompatible with the system environment, the functionality of the system may be affected.

The accuracy of the price prediction depends on the quality and representativeness of the training dataset used for the ML model. It is assumed that the training data reflects current real estate market conditions in the target geographic area. The availability and quality of input data also play an important role in the accuracy of results produced by the system.

3.External Interface Requirements

3.1User Interfaces

The PySide6 desktop application serves as the primary user interface of this system, organized into the following major sections:

Main Application Window: Divided into an Input Panel on the left and a Result Display Area on the right. The input panel contains all input fields, dropdowns, and the Predict Price button. The result area shows the predicted price, cost breakdown, and tabs for the summary view.

Price Prediction Display: The Prediction Result tab displays the total predicted property price in INR, prominently formatted with the material grade applied. A cost breakdown table shows structure, finishing, plumbing, and electrical costs as both absolute values and percentages.

Cost Summary View: The Cost Summary tab provides a visual representation of the cost distribution across construction categories, helping users understand how the total price is composed.

3.2Hardware Requirements

Component	Specification
Processor	Intel Core i5 or equivalent (2.0 GHz+ dual-core)
RAM	4 GB minimum (8 GB recommended)
Storage	1 GB free disk space minimum
Display	1366 × 768 resolution or higher
Operating System	Windows 10/11 (64-bit) or Ubuntu Linux 20.04+
Internet	Not required – system operates fully offline

Constraints and Assumptions

- The system shall be implemented using Python 3.10+ as the primary language.
- The desktop UI shall be built using PySide6 for cross-platform compatibility on Windows and Linux.
- All communication between frontend and backend shall use JSON format exclusively.
- The system shall not require an internet connection for any core functionality.
- It is assumed that users have access to a Windows or Linux desktop computer meeting the minimum hardware requirements.
- The accuracy of the price prediction depends on the quality and representativeness of the training dataset.
- It is assumed that the training data reflects current real estate market conditions in the target geographic region.
- The system is designed for residential properties only; commercial properties are not supported in the current version.

3.3 Software Interfaces

The PySide6 frontend communicates with the FastAPI backend via local HTTP requests. SQLite is used for local data persistence.

Library / Tool	Version	Purpose
Python	3.10+	Core development language
PySide6	6.x	Desktop GUI application framework (Qt6)
FastAPI	0.100+	Python REST API backend framework
Scikit-learn	1.3+	Linear Regression model training and prediction
NumPy	1.24+	Numerical computations for feature engineering
Pandas	2.0+	Data manipulation and model feature preparation
ReportLab	4.0+	PDF generation for price report export
Uvicorn	0.20+	ASGI server for running the FastAPI backend
pickle	(stdlib)	Model serialization and deserialization
Git / GitHub	—	Version control system

4. System Features

4.1 Machine Learning Price Prediction

4.1.1 Description: Predicts price using feature engineering and Linear Regression.

REQ-1: Apply 0.85x multiplier for Economy and 1.25x for Premium material grades.

REQ-2: Predict price within 3 seconds of input submission.

4.2 PDF Exporting

4.2.1 Description: Generates a summary of inputs and predictions in PDF format.

REQ-3: Complete report generation within 10 seconds.

5. Nonfunctional Requirements

5.1 Performance Requirements

ML model must maintain an R2 accuracy score > 0.80. UI must load within 5 seconds of launch.

5.2 Security Requirements

The system shall support user identity authentication with hashed passwords stored in the local SQLite database.

6. Project Plan

Team Members: Nakul Agrawal, Naman Mathe, Naman Patil.

Division of Work: Analysis (Naman P.), Design (Naman M.), Coding & Testing (Nakul.).

Appendix B1 – Glossary

Term	Definition
AI	Artificial Intelligence – systems that simulate human intelligence
ML	Machine Learning – AI technique for learning patterns from data
FastAPI	Modern Python web framework for building REST APIs
PySide6	Python binding for Qt6, used for desktop GUI development
UML	Unified Modeling Language – standardized software design notation
ER Diagram	Entity-Relationship Diagram – database structure visualization
JSON	JavaScript Object Notation – lightweight data interchange format
API	Application Programming Interface – service communication contract
BHK	Bedroom, Hall, Kitchen – Indian residential unit classification
Vastu	Vastu Shastra – Indian directional architectural science
Layout Model	ML model that generates optimal room placement from plot data
Price Prediction	ML model that estimates property market value
Constraint	Rule that a generated layout must satisfy (e.g., minimum room size)
Matplotlib	Python plotting library used for rendering 2D floor plans
SDS	Software Design Specification – this document
PK	Primary Key – unique identifier for a database record
REST	Representational State Transfer – web API architectural style

Appendix B2 – To Be Determined (TBD) List

#	Item	Status
1	Final ML algorithm selection (Random Forest vs XGBoost for price prediction)	TBD
2	Real estate price dataset source and preprocessing pipeline	TBD
3	Cloud deployment configuration (AWS / Azure / Render)	TBD
4	User authentication and login system for web version	TBD
5	3D visualization of generated floor plans	TBD
6	Mobile application version (Android / iOS)	TBD
7	Integration with CAD export formats (DXF / DWG)	TBD
8	Multi-city price prediction support with location zones	TBD
9	Real-time collaborative editing of floor plans	TBD